

Đối tượng phân tích: GitHub REST API

[\[https://docs.github.com/en/rest\]](https://docs.github.com/en/rest)(<https://docs.github.com/en/rest>)

Review các tiêu chí API của Github

I. Đặt tên các resources đều là danh từ

Nhìn chung, hệ thống API của GitHub được thiết kế rất chuẩn mực, bám sát nguyên tắc RESTful cơ bản. Hầu hết các endpoints đều sử dụng danh từ số nhiều (collection) và tận dụng triệt để các HTTP Methods (GET, POST, PATCH, DELETE) để mô tả hành động.

Các ví dụ tiêu biểu:

1. Lấy danh sách repository:

- **URL:** GET /user/repos
- **Đánh giá:** Tuân thủ tốt. Dùng method GET để lấy dữ liệu, resource /repos là danh từ số nhiều.

2. Tạo mới một Issue:

- **URL:** POST /repos/{owner}/{repo}/issues
- **Đánh giá:** Tuân thủ tốt. Hành động "tạo mới" được giao cho method POST, URL chỉ chứa danh từ tài nguyên (repos, issues).

3. Cập nhật thông tin Repository:

- **URL:** PATCH /repos/{owner}/{repo}
- **Đánh giá:** Tuân thủ tốt. Dùng PATCH để cập nhật một phần dữ liệu thay vì viết các động từ như /update-repo lên URL.

4. Xóa một Repository:

- **URL:** DELETE /repos/{owner}/{repo}
- **Đánh giá:** Tuân thủ tốt. Hành động xóa được thể hiện qua method DELETE.

Mặc dù tuân thủ tốt ở các thao tác CRUD cơ bản, nhưng trong một số nghiệp vụ phức tạp, GitHub đã vi phạm tiêu chí 01 khi sử dụng động từ trực tiếp trên URL. Hệ thống sử dụng cách thiết kế dạng RPC (Remote Procedure Call) thay vì RESTful thuần túy cho các nghiệp vụ này.

Vấn đề 1: Thao tác Merge một Pull Request

- **URL hiện tại của GitHub:** PUT /repos/{owner}/{repo}/pulls/{pull_number}/merge
- **Vi phạm:** Chứa động từ merge ở cuối URL.
- **Đề xuất giải pháp sửa chữa:**

- **Góc nhìn:** Bản chất của việc "merge" là thay đổi trạng thái (state) của một Pull Request từ trạng thái open sang trạng thái merged.
- **URL đề xuất:** PATCH /repos/{owner}/{repo}/pulls/{pull_number}

Vấn đề 2: Thao tác Hủy (Cancel) một GitHub Action Workflow đang chạy

- **URL hiện tại của GitHub:** POST /repos/{owner}/{repo}/actions/runs/{run_id}/cancel
- **Vi phạm:** Chứa động từ cancel ở cuối URL.
- **Đề xuất giải pháp sửa chữa:**
 - **Góc nhìn:** Tương tự như vấn đề 1, "hủy bỏ" thực chất là cập nhật trạng thái (status) của tài nguyên Workflow Run (lượt chạy).
 - **URL đề xuất:** PATCH /repos/{owner}/{repo}/actions/runs/{run_id}

Kết luận: Chung quy lại Github đã tuân thủ tốt phần lớn tiêu chí resources là danh từ, tuy nhiên vẫn có một số vi phạm khi đã đặt động từ vào trong URLs. Lý do là khi quá tuân thủ vào khuôn khổ thì có những thao tác sẽ rất dài và khó hiểu, ưu tiên trải nghiệm, sự dễ hiểu, sự thực dụng hơn là phải tuân thủ các quy tắc.

II. Naming nhất quán

Ý nghĩa: Đường dẫn sử dụng chữ thường và định dạng kebab-case; tham số truy vấn sử dụng định dạng snake_case; danh từ số nhiều được áp dụng cho các tập hợp tài nguyên.

Kết quả phân tích trên GitHub REST API:

- **Cấu trúc đường dẫn URL:**
 - Github tuân thủ quy tắc sử dụng chữ viết thường và gạch nối (kebab-case) cho toàn bộ các phân đoạn đường dẫn động hoặc tĩnh.
 - Ví dụ:
 - /user/repository-invitations
 - /repos/{owner}/{repo}/git/matching-refs
- **Tham số truy vấn:**
 - Các tham số lọc, sắp xếp hoặc phân trang đều tuân thủ định dạng snake_case.
 - Ví dụ:
 - per_page=30
 - sort=created & direction=desc
 - since=2026-01-01T00:00:00Z
- **Định danh tập hợp tài nguyên:**

- Các danh mục đại diện cho tập hợp đối tượng luôn ở dạng số nhiều. Khi thao tác trên một đối tượng cụ thể, ID hoặc tên định danh sẽ được nối tiếp phía sau.
- Ví dụ:
 - Danh sách repositories: GET /user/repos
 - Một repository cụ thể: GET /repos/{owner}/{repo}
 - Danh sách issues: GET /repos/{owner}/{repo}/issues
 - Một issue cụ thể: GET /repos/{owner}/{repo}/issues/{issue_number}

III. Status code đúng nghĩa

Ý nghĩa: Mỗi response sử dụng mã HTTP phản ánh chính xác ngữ cảnh. Nghiêm cấm trả về mã 200 OK kèm nội dung lỗi trong thân phản hồi.

Kết quả phân tích trên GitHub REST API:

HTTP Code	Ý nghĩa	Ví dụ GitHub API
200 OK	Yêu cầu thành công, có dữ liệu trả về	GET /user, PATCH /repos/{owner}/{repo}
201 Created	Khởi tạo tài nguyên mới thành công	POST /repos/{owner}/{repo}/issues
204 No Content	Thao tác thành công, không có nội dung body	DELETE /repos/{owner}/{repo}, PUT /user/starred/{owner}/{repo}
304 Not Modified	Dữ liệu chưa thay đổi (Cache validation)	Sử dụng header If-None-Match / ETag
401 Unauthorized	Xác thực thất bại / Thiếu Token	Gửi Yêu cầu không có header Authorization

403 Forbidden	Không có quyền hoặc vượt Rate Limit	Vượt quá giới hạn request/giờ hoặc không đủ quyền Admin
404 Not Found	Không tìm thấy tài nguyên	Yêu cầu tài nguyên không tồn tại (hoặc ẩn Private Repo)
422 Unprocessable	Dữ liệu gửi lên không hợp lệ	Đẩy payload JSON thiếu trường bắt buộc khi tạo Issue

Nguyên tắc phân tách Response và Error Handling: GitHub không bao giờ trả về HTTP status 200 OK cho các giao dịch thất bại. Khi có lỗi phát sinh, HTTP status code luôn thuộc nhóm 4xx hoặc 5xx, đồng thời body trả về cấu trúc lỗi chuẩn rõ ràng:

```
{
  "message": "Validation Failed",
  "errors": [
    {
      "resource": "Issue",
      "field": "title",
      "code": "missing_field"
    }
  ],
  "documentation_url": "https://docs.github.com/rest/issues/issues#create-an-issue"
}
```

KẾT LUẬN

Qua quá trình rà soát chi tiết, GitHub REST API đáp ứng 100% các yêu cầu đặt ra tại tiêu chí 2 và tiêu chí 3:

- Thiết kế nhất quán về mặt ngữ nghĩa và hình thức đặt tên (Naming) giúp lập trình viên dễ tích hợp.
- Xử lý mã trạng thái HTTP chuẩn xác, phân định rõ ràng giữa thành công và thất bại ở mức giao thức mạng, đảm bảo tính tin cậy cao.

IV. IDEMPOTENCY rõ ràng

Tiêu chí 04 yêu cầu API phải thể hiện rõ ràng tính Idempotent (gọi N lần cho cùng một kết quả trên server như gọi 1 lần), kiểm soát chặt chẽ các tác vụ thay đổi dữ liệu và tài liệu hóa minh bạch cho lập trình viên. GitHub REST API đạt được các điểm tích cực nổi bật sau:

1. Hiện thực hóa triết đề ngữ nghĩa Idempotent trên các HTTP Methods

- **Nhóm thao tác thiết lập trạng thái (PUT):**

GitHub sử dụng PUT chuẩn xác cho các hành vi gán trạng thái, thay thế toàn bộ hoặc tạo liên kết có tính bất định.

- *Thao tác Star Repository:* Endpoint PUT

`/user/starred/{owner}/{repo}` được thiết kế để đánh dấu yêu thích một kho mã nguồn. Dù client gửi request một lần hay thử lại (retry) nhiều lần do ngắt kết nối mạng, server vẫn đảm bảo trạng thái cuối cùng là repository đã được star và phản hồi đồng nhất mã 204 No Content.

- *Thao tác gán quyền cộng tác viên:* Endpoint PUT

`/repos/{owner}/{repo}/collaborators/{username}` cho phép thiết lập quyền hạn cho thành viên. Việc gọi lặp lại cùng một payload cấu hình không làm sinh ra bản ghi dư thừa hay thay đổi quyền ngoài ý muốn.

- **Nhóm thao tác hủy bỏ và xóa tài nguyên (DELETE):**

GitHub tuân thủ nghiêm ngặt nguyên tắc triệt tiêu tài nguyên của DELETE.

- *Thao tác Unstar Repository:* DELETE `/user/starred/{owner}/{repo}` trả về 204 No Content. Trạng thái của kho mã nguồn sau lệnh gọi là không còn nằm trong danh sách starred của người dùng, bất kể số lần retry của client.

- *Xóa bình luận/tài nguyên con:* DELETE

`/repos/{owner}/{repo}/issues/comments/{comment_id}` đảm bảo tài nguyên bị gỡ bỏ hoàn toàn khỏi hệ thống, tránh mọi hiệu ứng phụ ngoài ý muốn khi có retry tự động.

- **Đảm bảo tính an toàn (Safe) tuyệt đối cho các thao tác đọc (GET, HEAD):**

Các endpoint truy vấn dữ liệu như `GET /repos/{owner}/{repo}/issues` hoàn toàn là thao tác chỉ đọc (read-only), không làm biến đổi trạng thái server (safe method), cho phép hệ thống mạng và gateway tự do kích hoạt cơ chế retry và caching mà không lo ngại rủi ro sai lệch dữ liệu.

2. Kiểm soát đồng thời và tính toàn vẹn dữ liệu qua Conditional Requests

- GitHub tích hợp cơ chế kiểm soát tương tranh lạc quan (Optimistic Concurrency Control) thông qua các HTTP Header tiêu chuẩn như ETag, If-Match và If-None-Match.

- Đối với các thao tác cập nhật nội dung quan trọng (chẳng hạn cập nhật file qua PUT `/repos/{owner}/{repo}/contents/{path}`), API yêu cầu bắt buộc cung cấp mã băm (sha) của commit hiện tại. Cơ chế này đảm bảo tính toàn vẹn trạng thái: nếu có nhiều request can thiệp cùng lúc hoặc

client gửi lại một request cũ, server sẽ ngăn chặn xung đột và bảo vệ nhánh dữ liệu chính xác.

3. Tài liệu hóa minh bạch và khả năng dự đoán cao (High Predictability)

- Toàn bộ tài liệu chính thức của GitHub phân định rõ ràng phương thức HTTP, trạng thái tài nguyên sau khi thực thi, và bảng mã phản hồi chuẩn hóa đi kèm cho từng kịch bản gọi lặp lại.
- Điều này giúp các thư viện SDK (Octokit) và hệ thống tích hợp phía client xây dựng chính xác logic tự động thử lại (retry strategy) mà không gây rủi ro sai lệch dữ liệu nghiệp vụ.

V. Error Response có cấu trúc

Tiêu chí 05 yêu cầu phản hồi lỗi phải có cấu trúc rõ ràng, cung cấp đầy đủ ngữ cảnh để định danh, giải thích và định vị lỗi mà không che giấu trạng thái HTTP thực tế. GitHub REST API đáp ứng xuất sắc các khía cạnh sau:

1. Cấu trúc JSON báo lỗi thống nhất trên toàn bộ hệ sinh thái

GitHub chuẩn hóa một định dạng phản hồi lỗi duy nhất cho toàn bộ các mã trạng thái 4xx và 5xx trên hàng trăm endpoints, giúp client dễ dàng viết hàm xử lý ngoại lệ tập trung (centralized error handling):

JSON

```
{
  "message": "Validation Failed",
  "errors": [
    {
      "resource": "Issue",
      "field": "title",
      "code": "missing_field"
    }
  ],
  "documentation_url":
    "https://docs.github.com/rest/issues/issues#create-an-issue"
}
```

- **Ý nghĩa ngữ nghĩa chặt chẽ tương đương chuẩn hiện đại:**
 - **Trường message:** Đóng vai trò là thông điệp tổng quan, súc tích dành cho con người đọc hiểu ngay bản chất vấn đề (tương đương trường title và detail theo tư tưởng RFC 7807/9457).

- **Trường `documentation_url`:** Cung cấp trực tiếp URI tham chiếu tới tài liệu chính thức mô tả chi tiết lỗi và cách khắc phục (đáp ứng đúng vai trò định danh chuẩn mực của trường type trong RFC 7807).

2. Báo lỗi chi tiết ở cấp độ thuộc tính (Field-level Validation)

- Đối với các lỗi vi phạm quy tắc nghiệp vụ hoặc dữ liệu đầu vào không hợp lệ, GitHub cung cấp mảng `errors` với độ chi tiết cao:
 - `resource`: Chỉ rõ thực thể dữ liệu bị lỗi (ví dụ: `Issue`, `Repository`, `PullRequest`).
 - `field`: Tên trường cụ thể không đạt yêu cầu (ví dụ: `title`, `body`, `base`).
 - `code`: Mã lỗi định danh máy đọc được (machine-readable error code) như `missing_field`, `invalid`, `already_exists`, `unprocessable`.
- Cấu trúc này giúp các ứng dụng client tự động ánh xạ thông báo lỗi lên đúng ô nhập liệu trên giao diện người dùng (UI) một cách chính xác mà không cần bóc tách chuỗi thủ công.

3. Tương thích chuẩn xác với HTTP Status Codes

- **Tuyệt đối tránh Anti-pattern trả 200 kèm lỗi:** GitHub không bao giờ trả về HTTP status code 200 OK đi kèm cờ lỗi trong body (`{"success": false}`). Toàn bộ lỗi đều được phản ánh đúng qua lớp mã trạng thái HTTP chuẩn.
- **Phân định rõ ràng ngữ nghĩa từng mã lỗi:**
 - 401 Unauthorized: Trả về khi request thiếu token xác thực hoặc token hết hạn.
 - 403 Forbidden: Trả về khi người dùng đã xác thực nhưng không có quyền trên tài nguyên hoặc khi chạm giới hạn tần suất gọi API (Rate limit).
 - 404 Not Found: Áp dụng cho các tài nguyên không tồn tại, đồng thời dùng để che giấu sự tồn tại của các kho mã nguồn riêng tư nhằm bảo mật thông tin.
 - 409 Conflict: Trả về khi có xung đột về trạng thái commit hoặc xung đột phiên bản dữ liệu.
 - 422 Unprocessable Entity: Được áp dụng chuẩn mực khi payload JSON đúng cú pháp nhưng dữ liệu vi phạm ràng buộc nghiệp vụ (semantic validation).

4. Bổ sung siêu dữ liệu hỗ trợ giám sát và gỡ lỗi (Observability)

- Đi kèm với cấu trúc JSON lỗi, GitHub luôn cung cấp các HTTP Header hỗ trợ gỡ lỗi và kiểm soát vận hành:

- X-GitHub-Request-Id: Định danh duy nhất cho từng request, hỗ trợ truy vết log hệ thống (tracing) tương tự vai trò của instance trong RFC 7807.
- X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset: Giúp client chủ động quản lý lưu lượng và thời điểm kích hoạt lại request khi gặp lỗi 403 hoặc 429 Too Many Requests.

Bảng Tổng Hợp Kết Luận Tiêu Chí 4 & 5:

Tiêu chuẩn đánh giá	Hiện thực hóa tại GitHub REST API	Giá trị mang lại
Tiêu chí 4: Idempotency rõ ràng	• PUT và DELETE đảm bảo kết quả trạng thái bất biến qua nhiều lần gọi. • Kiểm soát tương tranh bằng sha và ETag. • Tài liệu hóa đầy đủ hành vi và mã phản hồi tương ứng.	An toàn khi hệ thống tự động retry sau sự cố mạng, bảo vệ tính toàn vẹn của cơ sở dữ liệu.

Tiêu chí 5: Error response có cấu trúc	• JSON lỗi đồng nhất (message, documentation_url, errors). • Cung cấp chi tiết trường vi phạm (resource, field, code). • Ánh xạ đúng 100% HTTP Status Codes (401, 403, 404, 409, 422). • Đi kèm header định danh X-GitHub-Request-Id.	Phía client dễ dàng bắt lỗi tự động, thân thiện với lập trình viên và tối ưu hóa khả năng giám sát (monitoring).
---	--	--

VI. Pagination rõ ràng

(Nguồn: [github](#), [github discussion](#))

Cơ chế: GitHub hỗ trợ cả cursor và offset, đúng với cách tiêu chí 06 mô tả ("cursor/offset"). Header Link là cơ chế chung cho cả hai, nên client có thể xử lý tương đối thống nhất.

Trong tài liệu của GitHub:

- per_page tối đa 100 (mặc định 30) và page mặc định 1.
- Điểm đặc trưng là header Link trong response. Header này chứa các liên kết với rel là prev, next, last, first, nên client không cần tự tính URL trang kế tiếp.

- Tham số `per_page` được tự động đưa vào các URL trong header Link, vì vậy chỉ cần đi theo `rel="next"` là giữ nguyên kích thước trang.
- Ví dụ trong tài liệu: với `issues?per_page=2` của repo `Spoon-Knife`, header Link chỉ có `next` và `last`, với `last` trở tới trang 7715. Vì đang ở trang đầu nên không có `prev` và `first`.

Trong discussion:

- Mô tả hai kiểu: `cursor (before/after)` và `page (page/per_page)`.
- Ví dụ `cursor` là endpoint audit log của enterprise. Header Link trả về URL chứa chuỗi `cursor` mã hóa, có `rel` là `next`, `prev`, `first`; trong các ví dụ đó không có `rel="last"`. Mỗi trang 100 mục.
- Ví dụ `page` là Search code, header có `next`, `prev`, `first`, `last`, và `per_page=50` làm `last` giảm từ 34 xuống 20.

Đánh giá:

- Không có tổng số bản ghi trong body; muốn biết có bao nhiêu trang thì đọc `rel="last"` trong header.
- Bài cộng đồng nói tối đa 100 mục mỗi trang, và có endpoint ngoại lệ như `events` không cho đặt số lượng tối đa. Điều này cho thấy tiêu chí "có giới hạn trên" không áp dụng đồng đều.
- **Hạn chế:** Kiểu phân trang khác nhau theo endpoint, có endpoint chỉ dùng `page`, có endpoint chỉ dùng `cursor`, có endpoint hỗ trợ cả hai và người dùng phải tự đoán từ response.

VII. Filter/Sort đa dạng

Tham số thay đổi theo từng endpoint. Với *List repository issues*:

- **Sort:** `sort` (`created`, `updated`, `comments`) và `direction` (`asc`, `desc`). Chỉ sắp xếp theo một trường.
- **Filter:** `state`, `assignee`, `creator`, `mentioned`, `labels`, `milestone`, `type`, `issue_field_values`, `since`.

Một số filter của vài Endpoint tiêu biểu:

Endpoint	Filter	Sort	Pagination

List pull requests	state, head, base	sort, direction	Có, tài liệu dẫn về bài "Using pagination"
List organization repositories	type (loại repo)	sort, direction	per_page, page
List commits	sha, path, author, since, until (một số phiên bản còn có committer)	Không có tham số sort	Có
Search (code, repos, issues...)	q (chuỗi truy vấn có qualifier)	sort, order	per_page, page

Đánh giá:

- **Filter theo trường chính: mạnh và đúng nghiệp vụ.** Mỗi endpoint có bộ lọc riêng phù hợp với tài nguyên (path, author, since/until cho commit; base, head cho PR). Search còn có ngôn ngữ truy vấn qualifier trong q, linh hoạt hơn mọi endpoint list thông thường.
- **Sort: một trường + chiều.** Chỉ có sort kèm direction/order, không có sort đa trường. Commits thì không có tham số sort.
- **Không nhất quán tên gọi:** Danh sách thường dùng direction, còn Search dùng order.

VIII. Authentication & Security

GitHub REST API áp dụng các tiêu chuẩn bảo mật hiện đại nhất trong thiết kế API, đảm bảo việc xác thực an toàn tuyệt đối và quản lý lưu lượng truy cập minh bạch.

1. Phương thức xác thực an toàn qua Header (Authentication)

Copy markdown

About authentication

Many REST API endpoints require authentication or return additional information if you are authenticated. Additionally, you can make more requests per hour when you are authenticated.

To authenticate your request, you will need to provide an authentication token with the required scopes or permissions. There are a few different ways to get a token: You can create a personal access token, generate a token with a GitHub App, or use the built-in `GITHUB_TOKEN` in a GitHub Actions workflow.

After creating a token, you can authenticate your request by sending the token in the `Authorization` header of your request. For example, in the following request, replace `YOUR-TOKEN` with a reference to your token:

```
curl --request GET \
  --url "https://api.github.com/octocat" \
  --header "Authorization: Bearer YOUR-TOKEN" \
  --header "X-GitHub-API-Version: 2026-03-10"
```

Note

In most cases, you can use `Authorization: Bearer` or `Authorization: token` to pass a token. However, if you are passing a JSON web token (JWT), you must use `Authorization: Bearer`.

Failed login limit

If you try to use a REST API endpoint without a token or with a token that has insufficient permissions, you will receive a `404 Not Found` or `403 Forbidden` response. Authenticating with invalid credentials will initially return a `401 Unauthorized` response.

After detecting several requests with invalid credentials within a short period, the API will temporarily reject all authentication attempts for that user (including ones with valid credentials) with a `403 Forbidden` response. For more information, see [Rate limits for the REST API](#).

IN THIS
About
Auth
Auth
Auth
Auth
Furth

- **Truyền Token qua HTTP Header:** GitHub bắt buộc gửi thông tin xác thực trong Header Authorization dạng Bearer <YOUR-TOKEN>.
- **Bảo mật URL:** Tuyệt đối không cho phép truyền Access Token qua tham số URL Query (?access_token=...) để ngăn ngừa nguy cơ rò rỉ Token qua hệ thống Server Log, lịch sử trình duyệt hoặc Web Proxies.
- **Đa dạng loại Token:** Hỗ trợ linh hoạt Personal Access Tokens (PAT - bao gồm fine-grained và classic), GitHub App Installation Tokens và OAuth Tokens.

2. Quản lý giới hạn truy cập minh bạch (Rate Limit)

Checking the status of your rate limit [↗](#)

You can use the headers that are sent with each response to determine the current status of your primary rate limit.

Header name	Description
<code>x-ratelimit-limit</code>	The maximum number of requests that you can make per hour
<code>x-ratelimit-remaining</code>	The number of requests remaining in the current rate limit window
<code>x-ratelimit-used</code>	The number of requests you have made in the current rate limit window
<code>x-ratelimit-reset</code>	The time at which the current rate limit window resets, in UTC epoch seconds
<code>x-ratelimit-resource</code>	The rate limit resource that the request counted against. For more information about the different resources, see REST API endpoints for rate limits .

You can call the `GET /rate_limit` endpoint for a periodic overview of all resource families for the authenticated user. Calling this endpoint does not count against your primary rate limit, but it can count

- GitHub cung cấp cơ chế theo dõi Rate Limit cực kỳ chi tiết cho phía Client thông qua bộ Response Headers trả về trong mọi request:
 - `x-ratelimit-limit`: Số lượng request tối đa được phép thực hiện trong 1 giờ.
 - `x-ratelimit-remaining`: Số lượng request còn lại trong cửa sổ thời gian hiện tại.
 - `x-ratelimit-used`: Số lượng request đã sử dụng trong khung giờ.
 - `x-ratelimit-reset`: Thời điểm (định dạng UTC Epoch seconds) mà Rate Limit sẽ khôi phục lại ban đầu.
 - `x-ratelimit-resource`: Phân vùng tài nguyên đang chịu giới hạn (ví dụ: core, search, graphql).

3. Xử lý phản hồi lỗi bảo mật chuẩn xác

- **Xác thực thất bại**: Trả về HTTP Status Code 401 Unauthorized khi token không hợp lệ hoặc hết hạn.
- **Cảnh báo Spam/Lạm dụng**: Khi thực hiện quá nhiều request xác thực sai liên tiếp (Failed login limit) hoặc vượt quá Rate Limit, API lập tức phản hồi mã 403 Forbidden hoặc 429 Too Many Requests.

IX. Versioning + Deprecation

GitHub có chiến lược phân phiên bản độc đáo dựa trên ngày tháng (Date-based Versioning) truyền qua header, giúp duy trì tính ổn định của đường dẫn URL và cam kết lộ trình ngưng hỗ trợ rất minh bạch.

1. Cơ chế phân phiên bản theo Header (Header-based Versioning)

Specifying an API version [↗](#)

You should use the `X-GitHub-API-Version` header to specify an API version. For example:

```
curl --header "X-GitHub-API-Version:2026-03-10" https://api.github.com/zen
```

Requests without the `X-GitHub-API-Version` header will default to use the `2022-11-28` version.

If you specify an API version that is no longer supported, you will receive a `410 Gone` response.

Upgrading to a new API version [↗](#)

Before upgrading to a new REST API version, you should read the changelog of breaking changes for the new API version to understand what breaking changes are included and to learn more about how to upgrade to that specific API version. For more information, see [Breaking changes](#).

When you update your integration to specify the new API version in the `X-GitHub-API-Version` header, you'll also need to make any changes required for your integration to work with the new API version.

Once your integration is updated, test your integration to verify that it works with the new API version.

- **Date-based Versioning:** Thay vì đặt phiên bản trực tiếp lên URI (như `/v1/`, `/v2/`), GitHub sử dụng định dạng ngày phát hành `YYYY-MM-DD` để định danh phiên bản.
- **Cú pháp Header:** Developer chỉ định phiên bản bằng cách gửi kèm Header:
`X-GitHub-API-Version: 2022-11-28`
- **Phiên bản mặc định:** Nếu request không gửi header `X-GitHub-API-Version`, API sẽ tự động áp dụng phiên bản ổn định mặc định là `2022-11-28`.
- **Xử lý phiên bản hết hạn:** Nếu người dùng gửi yêu cầu với một phiên bản không còn được hệ thống hỗ trợ, API sẽ phản hồi chính xác bằng mã HTTP Status `410 Gone`.

2. Quy định thay đổi & Lộ trình ngưng hỗ trợ (Deprecation Policy)

Breaking changes are changes that can potentially break an integration. Breaking changes will be released in a new API version. We will provide advance notice before releasing breaking changes. Breaking changes include:

- Removing an entire operation
- Removing or renaming a parameter
- Removing or renaming a response field
- Adding a new required parameter
- Making a previously optional parameter required
- Changing the type of a parameter or response field
- Removing enum values
- Adding a new validation rule to an existing parameter
- Changing authentication or authorization requirements

Any additive (non-breaking) changes will be available in all supported API versions. Additive changes are changes that should not break an integration. Additive changes include:

- Adding an operation
- Adding an optional parameter
- Adding an optional request header
- Adding a response field
- Adding a response header
- Adding enum values

When a new REST API version is released, the previous API version will be supported for at least 24 more months following the release of the new API version.

For more information about API versions, see [API Versions](#).

- **Phân loại thay đổi hệ thống rõ ràng:**
 - *Breaking Changes (Thay đổi gây lỗi):* Bao gồm việc xóa một operation, xóa/đổi tên tham số hoặc trường dữ liệu trả về, yêu cầu thêm tham số bắt buộc mới, hoặc thay đổi yêu cầu xác thực. Các thay đổi này chỉ được áp dụng khi phát hành API Version mới.
 - *Additive Changes (Thay đổi bổ sung):* Bao gồm thêm operation, thêm optional parameter, thêm response field/header mới. Các thay đổi này được cập nhật ngay trên tất cả các phiên bản đang được hỗ trợ.
- **Cam kết thời gian hỗ trợ (Deprecation Commitment):** Khi một phiên bản REST API mới được phát hành, phiên bản API cũ trước đó được GitHub cam kết duy trì hoạt động trong thời gian ít nhất 24 tháng. Khoảng thời gian này giúp các bên tích hợp có đủ thời gian chuyển đổi mà không gặp rủi ro gián đoạn dịch vụ.
"When a new REST API version is released, the previous API version will be supported for at least 24 more months following the release of the new API version."